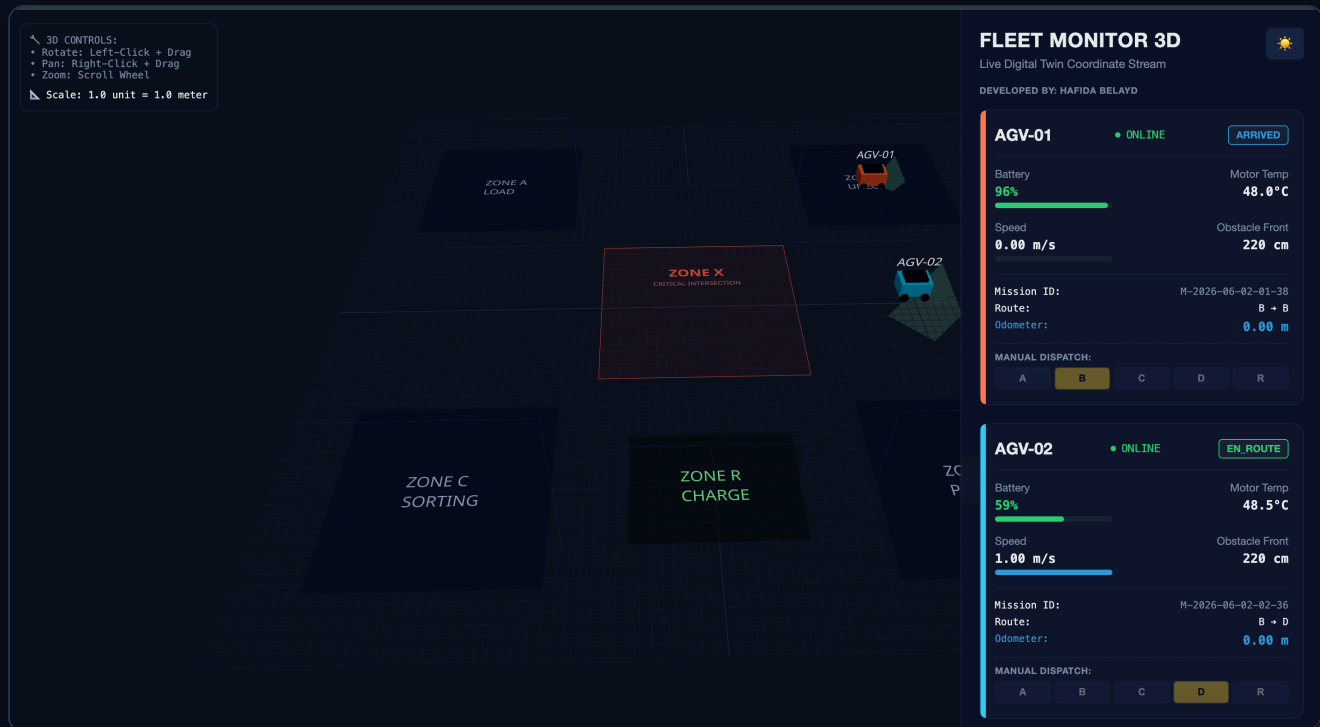


Documentation Jour 2 - AGV Fleet Simulator



1. Objectif de l'Étape 2

Projet : Physical AI - ABA Fusion | Auteur : Hafida Belayd



Objectif du Jour 2 : Cette mise à jour introduit de nouvelles mécaniques avancées pour la flotte d'AGV (Automated Guided Vehicles), notamment la conduite différentielle (Differential Drive), la gestion intelligente des collisions avec des priorités de passage, l'intégration complète avec du matériel physique (Arduino), et une refonte de l'interface utilisateur avec l'ajout du mode Clair/Sombre.

1. Conduite Différentielle (Differential Drive Mechanics)

Afin de simuler le comportement réel des robots, les AGV utilisent désormais un modèle cinématique de conduite différentielle :

- **Rotation sur place** : Lorsqu'un AGV doit changer de direction vers un nouveau point, il s'arrête d'avancer (translation nulle) et effectue une rotation sur place jusqu'à s'aligner parfaitement avec sa cible.
- **Vitesses de roues opposées** : Pendant la rotation, les roues gauche et droite reçoivent des commandes inversées. Par exemple, pour tourner à droite, la roue gauche avance à 100% de vitesse (`motor_left_pct = 1.0`) et la roue droite recule à 100% (`motor_right_pct = -1.0`).
- Cette mise à jour assure que la télémétrie envoyée aux moteurs physiques correspond exactement aux manœuvres requises sur le terrain.

2. Gestion des Collisions et Priorité (Priority Yielding)

La détection d'obstacles a été améliorée pour intégrer des règles de circulation (Traffic Rules) lors des rencontres entre deux robots :

- **Détection frontale** : Les AGV utilisent un capteur de proximité virtuel (cône de vision de 45 degrés). Si la distance frontale avec un obstacle tombe sous la barre des 110 cm, le robot réagit.
- **Céder le passage** : Si les robots `AGV-01` et `AGV-02` se retrouvent face à face dans le même couloir, `AGV-02` (ayant la priorité la plus basse) passe en état `YIELDING` (Céder le passage) et s'arrête, laissant à `AGV-01` l'opportunité de passer.

3. Interface Utilisateur 3D (Thème Clair / Sombre)

Le jumeau numérique 3D (Digital Twin), développé avec **React Three Fiber**, propose désormais une interface adaptative :

- **Bouton Bascule (Toggle)** : Un bouton  a été ajouté au tableau de bord. Il permet de passer instantanément du mode Sombre (Dark Mode) au mode Clair (Light Mode).
- **Mise à jour dynamique de l'environnement** : Le changement de thème affecte à la fois les couleurs du HUD (tableau de bord) et l'environnement 3D (intensité des lumières, couleur du brouillard, et couleur de la grille du sol).

4. Intégration du Matériel Physique (Hardware Integration)

Le projet ne se limite plus à la simulation logicielle ; il est maintenant un véritable système cyber-physique (Cyber-Physical System).

4.1 Script Python de Pont (Bridge)

Le fichier `physical_agv_controller.py` se connecte au flux WebSocket du simulateur (port `8765`). Il extrait les vitesses des moteurs (`m1`, `m2`, `m3`, `m4`) calculées en temps réel et les envoie via le port Série (USB) à l'Arduino.

4.2 Configuration Arduino (C++)

Le script `arduino_agv.ino` pilote directement les contrôleurs de moteurs (type TB717A3) selon les spécifications suivantes :

- **AGV-01 (Driver A)** : Moteur Gauche (PWMA: `3`, AIN1: `2`, AIN2: `4`) | Moteur Droit (PWMA: `5`, AIN1: `A0`, AIN2: `A1`).
- **AGV-02 (Driver B)** : Moteur Gauche (PWMA: `9`, AIN1: `11`, AIN2: `10`) | Moteur Droit (PWMA: `6`, AIN1: `12`, AIN2: `13`).

Traitement du Signal : L'Arduino convertit automatiquement les vitesses négatives (venant de la commande de rotation sur place) en inversant la polarité (broches de direction `HIGH` / `LOW`) et en appliquant la valeur absolue pour le signal PWM.

4.3 Affichage LCD I2C

Un écran LCD 16x2 est connecté en I2C (adresse `0x27`) à l'Arduino. Il affiche en temps réel les vitesses actuelles et l'état de la batterie des deux robots, fournissant un retour visuel direct sur le matériel :

```
R1 S:255  B:80 %  
R2 S:0    B:100%
```

5. Architecture Logique et Algorithmique

Le cœur du simulateur repose sur plusieurs algorithmes et modèles logiques assurant une simulation réaliste et autonome :

5.1 Machine à États Finis (State Machine)

Chaque AGV est régi par un automate fini composé des états suivants :

- **IDLE** : Le robot est inactif et attend une nouvelle mission. Il planifie automatiquement une destination s'il est laissé libre.
- **EN_ROUTE** : L'AGV se déplace vers son prochain nœud ou effectue une rotation sur place. La batterie se décharge et la température du moteur augmente.
- **WAIT** : Attente active. Ce statut est déclenché si l'accès à la zone d'intersection (Zone X) est bloqué par un autre robot (Mutex) ou s'il est en cours de recharge.
- **YIELDING** : État spécifique d'évitement de collision. L'AGV cède la priorité à un autre AGV sur une trajectoire frontale.
- **STOP** : Arrêt d'urgence (E-Stop) ou arrêt de sécurité si un obstacle est détecté à très courte distance.
- **ARRIVED** : L'AGV a atteint sa destination finale et marque une pause avant de repasser en **IDLE**.

5.2 Navigation et Algorithme de Dijkstra

Le routage est basé sur la théorie des graphes. Les zones (A, B, C, D, R, X) sont représentées comme des nœuds connectés par des arêtes pondérées. L'algorithme de **Dijkstra** (via `find_shortest_path`) calcule le chemin optimal. Le robot décompose ensuite ce chemin en segments rectilignes, calculant son orientation avec la fonction trigonométrique `math.atan2(dy, dx)` pour déterminer l'angle cible et ajuster la rotation différentielle.

5.3 Contrôleur de Trafic et Exclusion Mutuelle (Mutex)

L'intersection centrale (Zone X) agit comme un goulot d'étranglement. Pour éviter les collisions, la classe `ZoneXTrafficController` implémente un mécanisme d'exclusion mutuelle (Mutex) :

- Avant d'entrer dans le nœud 'X', un AGV appelle `request_entry()`.

- Si le verrou est libre, l'AGV obtient le passage (`has_zone_x_lock = True`). Sinon, il passe en état `WAIT` .
- Une fois le nœud 'X' franchi, l'AGV appelle `release_zone()` , libérant l'intersection pour les autres.

5.4 Simulation Physique et Énergétique

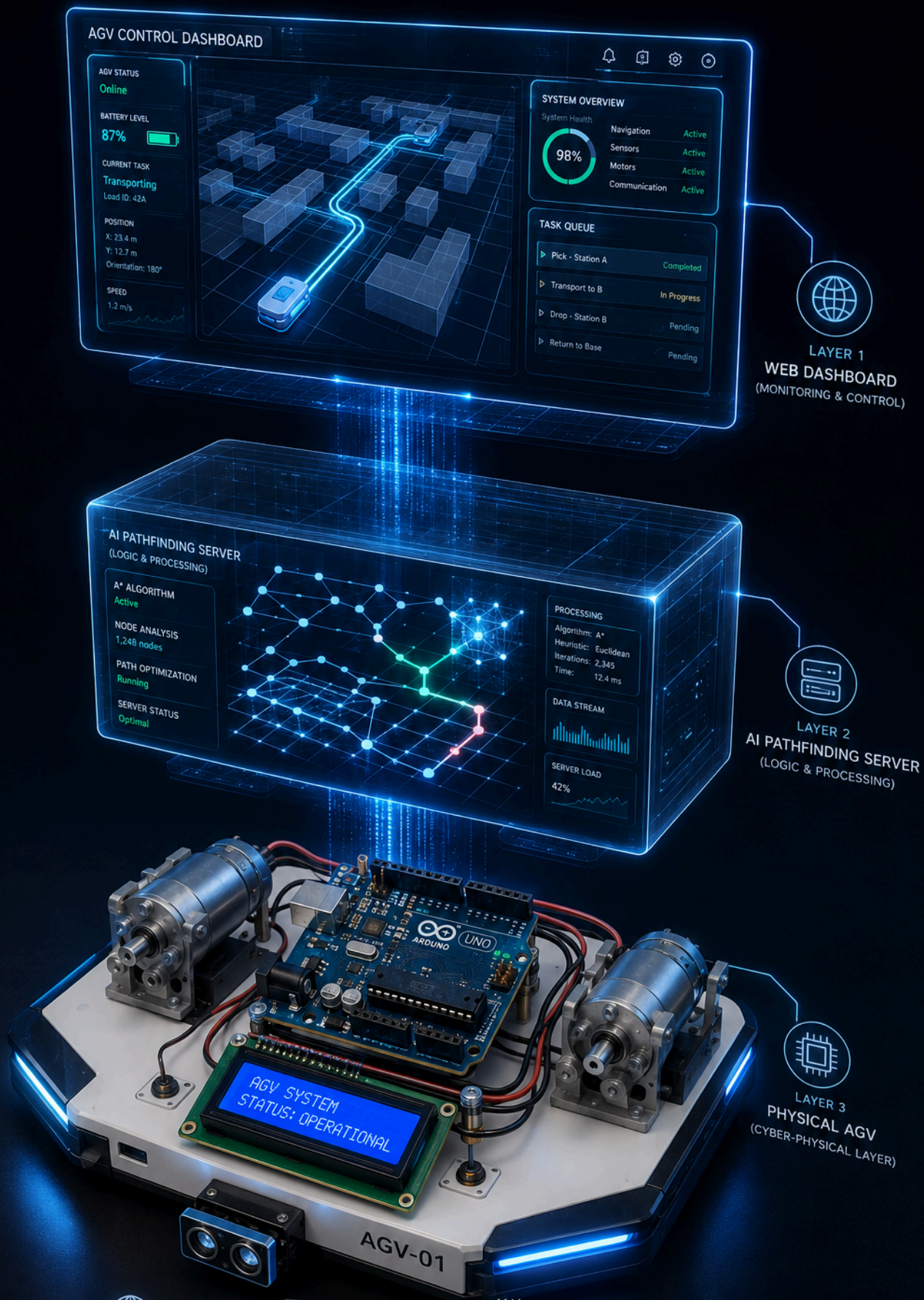
Le jumeau numérique calcule continuellement la physique des composants à une fréquence de 10 Hz (Delta Time `dt`) :

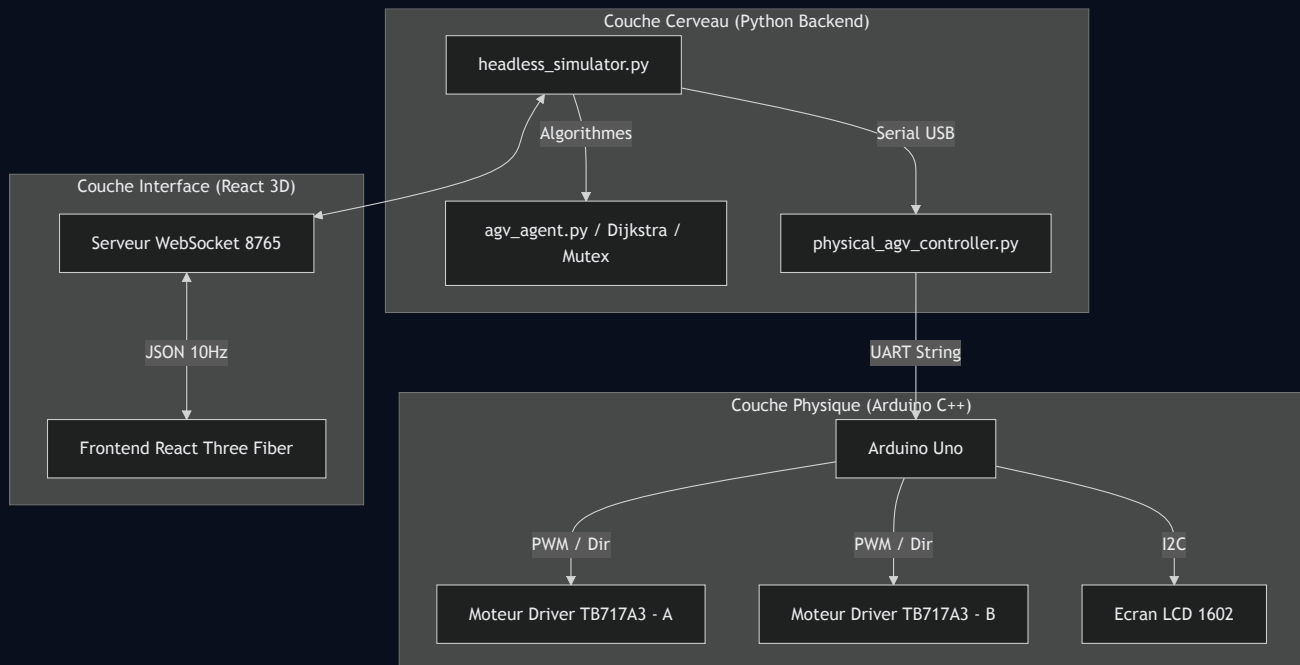
- **Batterie** : Diminue de 0.5% par seconde en mouvement, et se recharge de 8% par seconde lorsqu'il est en station (Zone R). En dessous de 20%, l'AGV planifie un retour d'urgence à la station.
- **Thermique** : La température du moteur augmente en charge (max 48.5°C) et diminue au repos (min 24.0°C), ajoutant un léger bruit mathématique (`random.uniform`) pour simuler les fluctuations réelles des capteurs.

6. Diagrammes d'Architecture et Flux

A. Diagramme d'Architecture Système

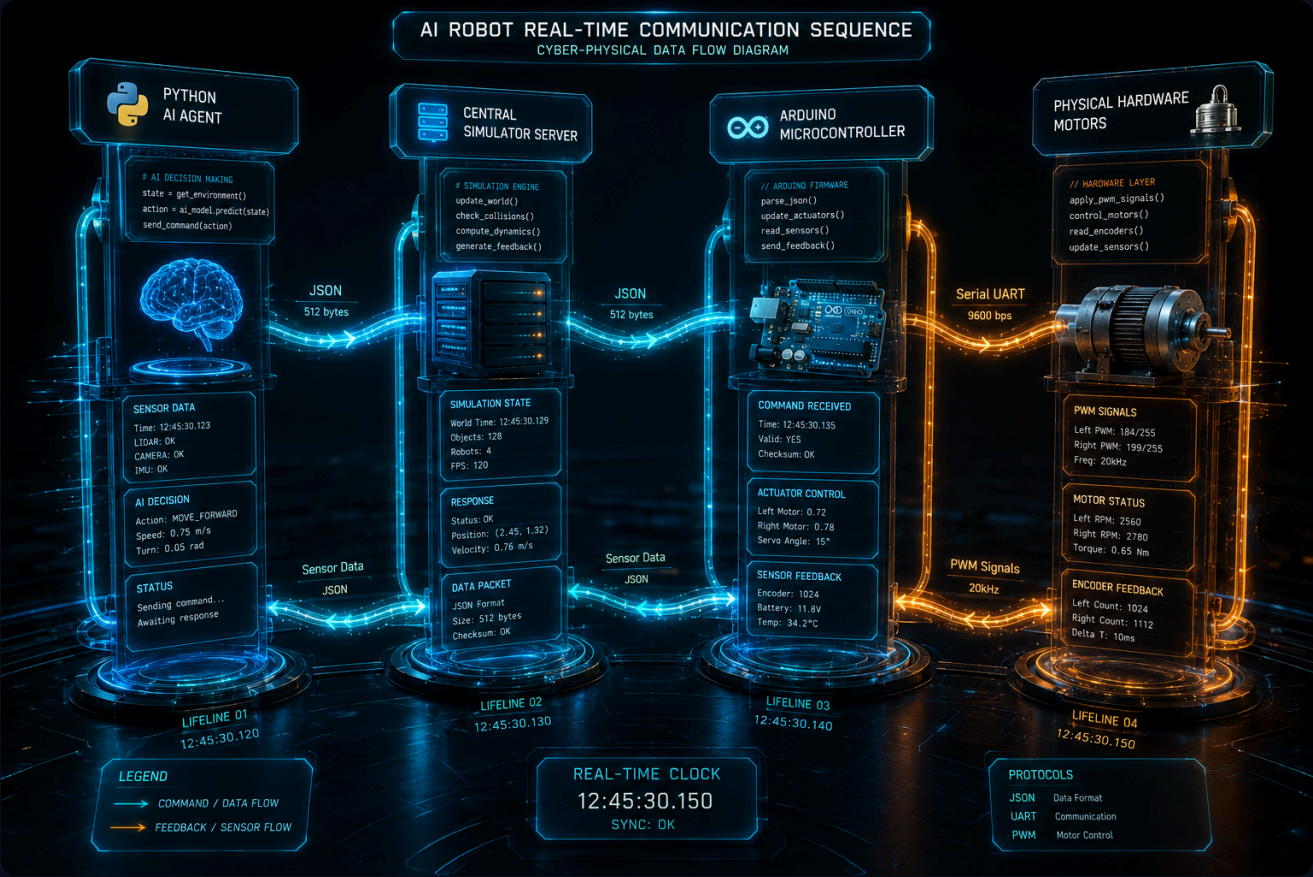
Le système est divisé en trois couches : la Simulation (Cerveau), le Jumeau Numérique 3D (Interface), et le Matériel Physique (Actionneurs).

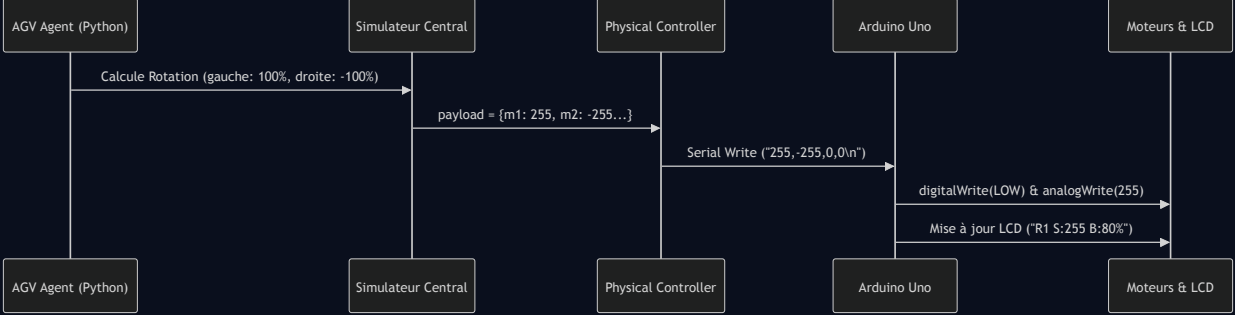




B. Diagramme de Flux de Données (Séquence)

Ce diagramme illustre le flux en temps réel, de la prise de décision logique jusqu'à la rotation physique des roues.





7. Tableau Récapitulatif des Protocoles de Communication

Protocole	Composants Connectés	Fréquence / Vitesse	Type de Données
WebSocket	Simulateur Python ↔ React 3D Frontend	10 Hz (Toutes les 0.1s)	JSON (Télémétrie complète, Positions, Batterie)
Serial (UART)	Python Bridge ↔ Arduino Uno	9600 Bauds	String CSV ("m1,m2,m3,m4\n")
I2C	Arduino Uno ↔ Écran LCD 1602	Sur Changement d'état	Texte (16 caractères par ligne)
PWM & GPIO	Arduino Uno ↔ Drivers Moteurs (TB717A3)	Continu	Signaux Électriques (0-255 Analog, HIGH/LOW Digital)

8. Registre des Risques et Impacts Analytiques

Risque Identifié	Niveau	Impact sur le Système	Mitigation Actuelle / Raison
Absence de Capteur de Batterie Réel	Moyen	L'écran LCD affiche une valeur fictive fixe. Le simulateur ne peut pas savoir si la batterie physique est réellement vide, ce qui pourrait causer un arrêt inattendu du robot physique.	Pour le moment, les valeurs de batterie ont été fixées en dur dans le C++ (<code>BATTERY_1_LEVEL = 80</code>) pour permettre les tests d'affichage sur le LCD 1602.
Désynchronisation Série (UART)	Faible	Les commandes moteurs peuvent arriver corrompues, entraînant un mouvement erratique.	Utilisation de <code>scanf</code> strict avec format <code>%d,%d,%d,%d</code> et d'un timeout très court (10ms) sur l'Arduino.
Boucle Ouverte (Pas de retour encodeur)	Élevé	Le jumeau numérique suppose que le robot physique a parfaitement effectué sa rotation différentielle. En réalité, le glissement des roues peut créer un décalage.	Assumé dans ce prototype. Le simulateur dicte la vérité absolue au détriment de l'odométrie physique.

9. Fiche de Recommandations Priorisées

1. **Intégration d'un Pont Diviseur de Tension (Urgence Haute)** : Remplacer les constantes matérielles de batterie par une lecture analogique (`analogRead`) de la batterie LiPo via un pont diviseur de tension, puis renvoyer cette valeur via Serial au simulateur Python.
2. **Encodeurs Optiques ou Magnétiques (Urgence Moyenne)** : Ajouter des encodeurs sur les roues pour implémenter un PID dans l'Arduino. Cela garantira que les vitesses `m1, m2` demandées par le simulateur correspondent exactement à la vitesse réelle (Boucle Fermée).
3. **Capteurs de Proximité Ultrason (HC-SR04)** : Bien que le simulateur calcule la "distance_front_cm" virtuellement, ajouter des capteurs réels permettrait à l'Arduino de stopper les moteurs indépendamment en cas de perte de connexion Serial.
4. **Baud Rate** : Augmenter la vitesse Serial de 9600 à 115200 bauds pour réduire la latence de commande entre le simulateur et l'Arduino.